

PySpark Cheat Sheet

Quick Reference Guide for Data Engineering & Manipulation

1. SPARKSESSION & READING DATA

The entry point to PySpark functionality and structured data loading.

```
from pyspark.sql import SparkSession

# Initialize Spark Session
spark =
SparkSession.builder      .appName("QuickRef")

# Read CSV
df = spark.read.csv("data.csv", header=True,
inferSchema=True)

# Read Parquet (Highly Recommended)
df = spark.read.parquet("data.parquet")

# Read JSON
df = spark.read.json("data.json")
```

2. INSPECTING DATA

Quick methods to check structure, columns, and sample counts.

```
df.show(5)           # Display first 5 rows
df.printSchema()     # Print column types &
structure
df.columns           # Get list of column
names
df.count()           # Get total row count
df.describe().show() # Summary statistics
```

4. AGGREGATIONS & GROUPING

```
df.groupBy("department").agg(
    F.sum("salary").alias("total_salary"),
    F.avg("salary").alias("avg_salary"),

    F.count("employee_id").alias("emp_count")
).show()
```

5. JOINS

Always explicit join types to avoid automatic inner joins.

```
# Standard Inner Join
res = df1.join(df2, on="user_id",
how="inner")

# Left Join with explicit column mappings
res = df1.join(df2, df1.id == df2.user_id,
how="left")
```

6. MISSING DATA (NULLS)

```
# Drop rows with any null fields
df.na.drop()

# Drop subset nulls
df.na.drop(subset=["email", "phone"])

# Fill missing values with constant mapping
df.na.fill({"status": "Unknown", "age": 0})

# Conditional If-Else Statement
df = df.withColumn("clean",
    F.when(F.col("status").isNull(),
"Missing")
    .otherwise(F.col("status")))

```

3. BASIC TRANSFORMATIONS

Transformations are **lazy** and build an execution plan.

```
from pyspark.sql import functions as F

# Select specific columns
df.select("col1", "col2")

# Rename a column
df = df.withColumnRenamed("old_name",
"new_name")

# Add / Modify a column
df = df.withColumn("new_col", F.col("col1")
* 10)

# Filter rows
df = df.filter(F.col("age") > 21)
df = df.where((F.col("age") > 21) &
(F.col("status") == "Active"))
```

7. WINDOW FUNCTIONS

```
from pyspark.sql.window import Window

# Define analytical frame partition
wSpec =
Window.partitionBy("dept").orderBy(F.col("salary"))

# Apply window rankings
df = df.withColumn("rank",
F.rank().over(wSpec)) .withColumn("row_number",
F.row_number().over(wSpec))
```

8. PERFORMANCE & WRITING

```
# Broadcast smaller tables to bypass
shuffling
from pyspark.sql.functions import broadcast
df_large.join(broadcast(df_small), "id")

# Cache reusable states
df.cache()

# Partitioning when saving to disk
df.write.mode("overwrite") .partitionBy("year")
```

⚠ Driver Memory Precaution

Avoid using `.toPandas()` or `.collect()` on production scale big-data sets. Doing so brings all cluster partitions down into a single master driver's node memory, easily triggering **Out Of Memory (OOM) Errors**. Always leverage native Spark SQL functions wherever possible.